Student Name and Number as per student card:

Hadi Omar 20028514

Programme:

Software Development

Lecturer Name:

Paul Laird

Module/Subject Title:

B8IT150 Advanced Programming

Assignment Title:

Electronics Store Website Development

By submitting this assignment, I am confirming that:

- This assignment is all my own work;
- Any sources used have been referenced;
- I have followed the Generative AI instructions/ scale set out in the Assignment Brief;
- I have read the College rules regarding academic integrity in the QAH Part B Section 3, and the Generative AI Guidelines, and understand that penalties will be applied accordingly if work is found not to be my/our own.
- I understand that all work submitted may be code-matched report to show any similarities with other work.

Introduction:

Project Overview:

This project involves the design and development of a fully functional e-commerce web application for an Electronics Store. The application aims to provide users with a modern, responsive, and interactive online platform to browse, search for, and purchase a variety of electronic products such as mobile phones, laptops, tablets, and accessories. Built using a combination of front-end and back-end web technologies, the project simulates a real-world online shopping experience with features such as user registration, login/logout, product catalog, search functionality, shopping cart, and checkout.

Project Objectives:

- To design a responsive, user-friendly front-end interface using HTML5, CSS3, Bootstrap, JavaScript, and jQuery.
- To implement back-end functionality using Node.js and Express to handle routing, server-side logic, and user authentication.
- To design and connect a MySQL database for storing user information, product details, and transaction records.
- To apply client-side and server-side validation techniques to ensure data integrity.
- To deliver a fully integrated e-commerce application that reflects industry-standard practices in web development.
- To provide documentation and a demonstration video showcasing the system’s functionality and development process.

Front-End Development:

Page – Add New Product:

Purpose:

This page allows administrators or sellers to add new products to the electronics store database. It includes a form for entering product details such as name, description, price, image URL, stock quantity, and category.

Key Features:

- **Responsive Layout:** Uses Bootstrap 5 grid system and utility classes for responsiveness and spacing.
- **Client-Side Validation:** Required fields such as Product Name, Price, and Category ensure users cannot submit incomplete forms.
- **Semantic HTML:** Clear labeling with <label> tags and appropriate input types (e.g., type="number", type="url").
- **Interactive Elements:** JavaScript handles the form submission using an asynchronous addProduct() function (likely defined in api.js).
- **UX Improvements:** Includes a cancel button that redirects to the welcome.html page, and a statusMsg div to display feedback messages.
- **Form Fields:**
    - Product Name – Text input
    - Description – Multi-line input
    - Price – Number input with decimal support
    - Image URL – URL input
    - Stock – Number input
    - Category – Dropdown with various accessory types

Technologies Used:

- **HTML5**: Structuring content
- **Bootstrap 5**: Responsive layout and styling

- **JavaScript**: Handles form validation and event handling

```
<!DOCTYPE html>

<html lang="en">



<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>Add Product - E-Commerce App</title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />

  <link rel="stylesheet" href="./css/styles.css" />

</head>



<body class="bg-light">

  <div id="navbar"></div>



  <div class="container py-5">

    <div class="row justify-content-center">

      <div class="col-md-6">

        <h3 class="mb-4">Add New Product</h3>



        <form id="addProductForm">

          <div class="mb-3">

            <label for="name" class="form-label">Product Name</label>
```

```
    <input type="text" id="name" class="form-control" required />
  </div>

  <div class="mb-3">
    <label for="description" class="form-label">Description</label>
    <textarea id="description" class="form-control" rows="3"></textarea>
  </div>

  <div class="mb-3">
    <label for="price" class="form-label">Price ($)</label>
    <input type="number" step="0.01" id="price" class="form-control" required />
  </div>

  <div class="mb-3">
    <label for="image_url" class="form-label">Image URL</label>
    <input type="url" id="image_url" class="form-control" />
  </div>

  <div class="mb-3">
    <label for="stock" class="form-label">Stock</label>
    <input type="number" id="stock" class="form-control" />
  </div>

  <div class="mb-3">
```

```
            <label for="category" class="form-label">Category</label>
            <select id="category" class="form-select" required>
              <option value="">-- Select Category --</option>
              <option value="Phone Accessories">Phone Accessories</option>
              <option value="Laptop Accessories">Laptop Accessories</option>
              <option value="Smartwatch Accessories">Smartwatch Accessories</option>
              <option value="Audio & Headphones">Audio & Headphones</option>
              <option value="Chargers & Cables">Chargers & Cables</option>
              <option value="Bags & Cases">Bags & Cases</option>
              <option value="Gaming Accessories">Gaming Accessories</option>
              <option value="Car Accessories">Car Accessories</option>
              <option value="Camera Accessories">Camera Accessories</option>
              <option value="Fashion Accessories">Fashion Accessories</option>
            </select>
          </div>

          <button type="submit" class="btn btn-success">Add Product</button>
          <a href="./welcome.html" class="btn btn-secondary ms-2">Cancel</a>
        </form>

        <div id="statusMsg" class="mt-3"></div>
      </div>
    </div>
  </div>
```

```
  <script src="./js/api.js"></script>
  <script>
     document.getElementById('addProductForm').addEventListener('submit', async function (e) {
         e.preventDefault();

         const name = document.getElementById('name').value.trim();
         const description = document.getElementById('description').value.trim();
         const price = parseFloat(document.getElementById('price').value);
         const image_url = document.getElementById('image_url').value.trim();
         const stock = parseInt(document.getElementById('stock').value);
         const category = document.getElementById('category').value;

         const productData = { name, description, price, image_url, stock, category };

         addProduct(productData);
     });

  </script>
</body>

</html>
```

Page – Your Cart:

Purpose:
This page displays the items a user has added to their shopping cart. It enables users to review selected products, adjust or remove them, and proceed to checkout.

Key Features:

- **Cart Display:** Dynamically lists all cart items using data retrieved from localStorage, showing:
    - Product name
    - Price
    - Quantity
    - Subtotal
- **Total Calculation:** Automatically calculates and displays the total cost of all items.
- **Buttons:**
    - **Clear Cart** – Empties the cart by calling removeAllItemsFromcart()
    - **Checkout** – Redirects to payment.html
    - **Remove** – Removes an individual item
    - **Order** – Simulates placing an order for a single item using an alert

Technologies Used:

- **HTML5** – Basic page structure
- **Bootstrap 5** – Layout, card design, and button styling
- **JavaScript** – Handles dynamic rendering of cart items, subtotal/total calculations, and interactions
- **Local Storage** – Retrieves cart data for persistent shopping experience

UX/Functionality Notes:

- Empty cart message appears when no items are present.
- Uses a loop to generate cart item elements dynamically.
- Modular and scalable structure for adding more cart functionality in the future.

```
<!DOCTYPE html>

<html lang="en">



<head>

    <meta charset="UTF-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <title>Your Cart - E-Commerce App</title>

    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />

</head>



<body class="bg-light">



    <div id="navbar"></div>



    <div class="container py-5">

        <h2>Your Cart</h2>

        <div class="d-flex justify-content-between my-3">

            <button class="btn btn-outline-danger btn-sm" onclick="clearCart()">Clear Cart</button>

            <button class="btn btn-success btn-sm" onclick="checkout()">Checkout</button>

        </div>
```

```
        <div id="cartItems" class="mt-4"></div>
        <div id="totalSection" class="mt-3 fw-bold"></div>
    </div>

    <script src="./js/api.js"></script>
    <script>
        document.addEventListener('DOMContentLoaded', function () {
            renderCart();
        });

        function renderCart() {
            const cart = JSON.parse(localStorage.getItem('cart')) || [];
            const cartItemsDiv = document.getElementById('cartItems');
            const totalSection = document.getElementById('totalSection');
            cartItemsDiv.innerHTML = '';

            if (cart.length === 0) {
                cartItemsDiv.innerHTML = '<p class="text-danger">Your cart is empty.</p>';
                totalSection.innerHTML = '';
                return;
            }

            let total = 0;
```

```
            cart.forEach((item, index) => {
                const subtotal = item.price * item.quantity;
                total += subtotal;

                const row = document.createElement('div');
                row.className = 'card mb-3 p-3';
                row.innerHTML = `
                    <div class="d-flex justify-content-between align-items-center">
                      <div>
                        <h5>${item.name}</h5>
                        <p class="text-muted">Price: $${item.price}</p>
                        <p class="text-muted">Quantity:
                          <span class="mx-2">${item.quantity}</span>
                        </p>
                        <p class="text-muted">Subtotal: $${subtotal.toFixed(2)}</p>
                      </div>
                      <div>
                        <button class="btn btn-danger btn-sm me-2"
onclick="removeCartItem(${item.id})">Remove</button>
                        <button class="btn btn-primary btn-sm"
onclick="orderItem(${item.id})">Order</button>
                      </div>
                    </div>
                  `;
```

```
                cartItemsDiv.appendChild(row);
            });

            totalSection.innerHTML = `Total: $${total.toFixed(2)}`;
        }

        function clearCart() {
            removeAllItemsFromcart()
        }

        function checkout() {
            window.location.href = "./payment.html"
        }

        function orderItem(id) {
            alert(`Order placed for item ID: ${id}`);
        }
    </script>
</body>

</html>
```

Page – Edit Product:

Purpose:

This page allows admin users to update the details of an existing product

within the electronics store system. It retrieves the current data for a product (based on the ID from the URL), pre-fills the form, and sends updated values to the backend when the form is submitted.

Key Features:

- **Product Retrieval:** Extracts the productId from the URL (?id=123) and fetches the product details using fetchProduct(), a function likely defined in api.js.
- **Pre-Filled Form:** Automatically populates the input fields with existing data using the fetched product object.
- **Form Submission:** Sends updated product details to the server via updateProduct().
- **Validation:** Required fields include Product Name, Price, and Category.
- **User Feedback:** Displays error messages using the #statusMsg element if a product ID is missing or the fetch fails.
- **Cancel Navigation:** Provides a cancel button linking back to the welcome/dashboard page.

Technologies Used:

- **HTML5**: Structuring the product form and interface
- **Bootstrap 5**: Styling form components and layout
- **JavaScript**: Handles data fetching, DOM manipulation, and form submission logic
- **URLSearchParams API**: Extracts query parameters for dynamic page behavior

UX/Functionality Notes:

- Ensures a smooth editing experience by auto-loading product details.
- Uses modern asynchronous JavaScript with async/await.
- Provides error handling for missing or failed data loads.

```
<!DOCTYPE html>

<html lang="en">
```

```
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Edit Product</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />
</head>

<body class="bg-light">
    <div id="navbar"></div>

    <div class="container py-5">
        <div class="row justify-content-center">
            <div class="col-md-6">
                <h3 class="mb-4">Edit Product</h3>

                <form id="editProductForm">
                    <div class="mb-3">
                        <label for="name" class="form-label">Product Name</label>
                        <input type="text" id="name" class="form-control" required />
                    </div>

                    <div class="mb-3">
```

```
                    <label for="description" class="form-label">Description</label>
                    <textarea id="description" class="form-control" rows="3"></textarea>
                </div>

                <div class="mb-3">
                    <label for="price" class="form-label">Price ($)</label>
                    <input type="number" step="0.01" id="price" class="form-control" required />
                </div>

                <div class="mb-3">
                    <label for="image_url" class="form-label">Image URL</label>
                    <input type="url" id="image_url" class="form-control" />
                </div>

                <div class="mb-3">
                    <label for="stock" class="form-label">Stock</label>
                    <input type="number" id="stock" class="form-control" />
                </div>

                <div class="mb-3">
                    <label for="category" class="form-label">Category</label>
                    <select id="category" class="form-select" required>
                        <option value="">-- Select Category --</option>
                        <option value="Phone Accessories">Phone Accessories</option>
```

```
                    <option value="Laptop Accessories">Laptop Accessories</option>
                    <option value="Smartwatch Accessories">Smartwatch Accessories</option>
                    <option value="Audio & Headphones">Audio & Headphones</option>
                    <option value="Chargers & Cables">Chargers & Cables</option>
                    <option value="Bags & Cases">Bags & Cases</option>
                    <option value="Gaming Accessories">Gaming Accessories</option>
                    <option value="Car Accessories">Car Accessories</option>
                    <option value="Camera Accessories">Camera Accessories</option>
                    <option value="Fashion Accessories">Fashion Accessories</option>
                  </select>
                </div>

                <button type="submit" class="btn btn-primary">Update Product</button>
                <a href="./welcome.html" class="btn btn-secondary ms-2">Cancel</a>
              </form>

              <div id="statusMsg" class="mt-3"></div>
            </div>
          </div>
        </div>

        <script src="./js/api.js"></script>
        <script>
          const productId = new URLSearchParams(window.location.search).get('id');
```

```
        const form = document.getElementById('editProductForm');
        const statusMsg = document.getElementById('statusMsg');

        async function populateForm() {
            const product = await fetchProduct(productId);
            if (!product.success) {
                statusMsg.innerHTML = `<div class="alert alert-danger">Failed to fetch product.</div>`;
                return;
            }

            document.getElementById('name').value = product.data.name || '';
            document.getElementById('description').value = product.data.description || '';
            document.getElementById('price').value = product.data.price || '';
            document.getElementById('image_url').value = product.data.image_url || '';
            document.getElementById('stock').value = product.data.stock || '';
            document.getElementById('category').value = product.data.category || '';
        }

        form.addEventListener('submit', async (e) => {
            e.preventDefault();

            const productData = {
                name: document.getElementById('name').value.trim(),
```

```
                description: document.getElementById('description').value.trim(),
                price: parseFloat(document.getElementById('price').value),
                image_url: document.getElementById('image_url').value.trim(),
                stock: parseInt(document.getElementById('stock').value),
                category: document.getElementById('category').value,
            };

            updateProduct(productId, productData);
        });

        // On load
        if (productId) {
            populateForm();
        } else {
            statusMsg.innerHTML = `<div class="alert alert-danger">No product ID specified in URL.</div>`;
        }
    </script>
</body>
</html>
```

Page – Login:

Purpose:

This page allows registered users to log into the e-commerce platform using their email and password credentials. It ensures proper input validation and securely handles login attempts using a back-end function.

Key Features:

- **User Authentication Form:**
    - Email input with built-in browser validation
    - Password input with required validation
- **Bootstrap Card Layout:** Centered login form using Bootstrap's vh-100 and card components for a clean, professional appearance.
- **Client-Side Validation:**
    - Uses required attributes and invalid-feedback elements
    - Prevents form submission with empty or invalid fields
- **Error Handling:**
    - Displays login error messages in #loginError if authentication fails
- **Navigation Link:**
    - Directs new users to the registration page (register.html)

Technologies Used:

- **HTML5:** Semantic form structure
- **Bootstrap 5:** Responsive styling, spacing, and card layout
- **JavaScript & jQuery:** Handles form submission logic
- **Modular API Logic:** Calls the loginUser() function (assumed in api.js) to process credentials

User Experience (UX) Notes:

- Clean and minimal UI encourages easy interaction
- Clearly labeled form inputs and helpful validation messages
- Form is visually centered and mobile-responsive

```
<!DOCTYPE html>

<html lang="en">
```

```
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Login - E-Commerce App</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />
    <link rel="stylesheet" href="./css/styles.css" />
</head>

<body class="bg-light">
    <div class="container d-flex justify-content-center align-items-center vh-100">
        <div class="card p-4 shadow rounded-4 w-100" style="max-width: 400px;">
            <h2 class="text-center mb-4">Login</h2>
            <form id="loginForm" novalidate>
                <div class="mb-3">
                    <label for="email" class="form-label">Email address</label>
                    <input type="email" class="form-control" id="email" required />
                    <div class="invalid-feedback">Please enter a valid email.</div>
                </div>
                <div class="mb-3">
                    <label for="password" class="form-label">Password</label>
                    <input type="password" class="form-control" id="password" required />
                    <div class="invalid-feedback">Password is required.</div>
```

```
                </div>
                <button type="submit" class="btn btn-primary w-100">Login</button>
                <div id="loginError" class="text-danger mt-2 text-center" style="display: none;"></div>
            </form>
            <p class="mt-3 text-center">
                New user? <a href="./register.html">Register here</a>
            </p>
        </div>
    </div>

    <script src="https://code.jquery.com/jquery-3.7.0.min.js"></script>
    <script src="./js/api.js"></script>

    <script>
        document.addEventListener("DOMContentLoaded", function () {
            const loginForm = document.getElementById('loginForm');
            loginForm.addEventListener('submit', function (e) {
                e.preventDefault();
                const email = document.getElementById('email').value;
                const password = document.getElementById('password').value;
                loginUser(email, password);
            });
        });
    </script>
```

```
    </body>

    </html>
```

Page – Product Details:

Purpose:

This page displays the full details of a specific product selected by the user from the catalog. It allows the user to view an enlarged product image, description, price, and add the product to their cart.

Key Features:

- **Dynamic Data Loading:**
    - Extracts the product ID from the URL (?id=123) and uses the fetchProduct() function to retrieve product data.
    - Displays a fallback message if the product is not found.
- **Image Gallery Interaction:**
    - Shows a main product image with optional thumbnail switching (currently supports a single image but scalable for multiple).
    - Clicking a thumbnail updates the main image via the setMainImage() function.
- **Product Info Display:**
    - Name, description, price, and a prominent “Add to Cart” button.
    - Uses addToCart() to handle cart functionality (likely defined in api.js).

Technologies Used:

- **HTML5** – Page structure
- **Bootstrap 5** – Grid system, layout spacing, buttons, and typography
- **JavaScript** – DOM manipulation, image interactivity, dynamic content rendering
- **URLSearchParams API** – Used to get the product ID from the query string

User Experience (UX) Notes:

- Clean and organized layout using a 2-column grid (col-md-6)
- Interactive image thumbnails enhance visual exploration
- Graceful error handling when no product data is found
- Redirect link (“Back to Catalog”) improves navigation

```
<!DOCTYPE html>
```

```
<html lang="en">

<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Product Details</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />
    <style>
        .thumbnail-img {
            width: 70px;
            height: 70px;
            object-fit: cover;
            cursor: pointer;
            border: 2px solid transparent;
        }

        .thumbnail-img.active {
            border-color: #007bff;
        }
    </style>
</head>

<body class="bg-light">
```

```
    <div id="navbar"></div>

    <div class="container py-5">
        <a href="./welcome.html" class="btn btn-outline-secondary mb-4">← Back to Catalog</a>

        <div id="statusMsg"></div>
        <div class="row" id="product-details">
            <!-- Product details will be injected here -->
        </div>
    </div>

    <script src="./js/api.js"></script>
    <script>
        const params = new URLSearchParams(window.location.search);
        const productId = params.get('id');
        document.addEventListener("DOMContentLoaded", async function () {
            const res = await fetchProduct(productId);
            if (!res.success || !res.data) {
                document.getElementById('product-details').innerHTML = `<p
class="text-danger">Product not found.</p>`;
                return;
            }

            const product = res.data;
```

```
            const images = [product.image_url];


            document.getElementById('product-details').innerHTML = `
          <div class="col-md-6">
            <img id="mainImage" src="${images[0]}" class="img-fluid rounded shadow-sm mb-3" alt="${product.name}">
            <div class="d-flex gap-2 flex-wrap">
              ${images.map((img, i) => `
                <img src="${img}" class="thumbnail-img ${i === 0 ? 'active' : ''}" onclick="setMainImage(this, '${img}')">
              `).join('')}
            </div>
          </div>
          <div class="col-md-6">
            <h2>${product.name}</h2>
            <p class="text-muted">${product.description || 'No description provided.'}</p>
            <h4 class="text-primary">$${product.price}</h4>
             <div class="mt-4">
              <button class="btn btn-success" onclick='addToCart(${JSON.stringify(product)})'>Add to Cart</button>
            </div>
          </div >
        `;
          });
```

```
        const setMainImage = (el, src) => {

            document.getElementById('mainImage').src = src;

            document.querySelectorAll('.thumbnail-img').forEach(img =>
    img.classList.remove('active'));

            el.classList.add('active');

        }


    </script>

</body>


</html>
```

Page – Payment:

Purpose:

This page finalizes the purchase process by collecting billing information and confirming payment for items in the user's shopping cart. It combines address input, payment method selection, and a dynamic cart summary.

Key Features:

1. Billing Address Form

- Captures essential details:

    - Full Name
    - Address
    - City
    - Zip Code
- All fields are required for form validation.

2. Payment Method

- Users can choose between **Credit Card** and **PayPal**.
- Implemented using radio buttons with a default selection.

3. Order Summary

- Dynamically lists cart items from localStorage.
- Calculates and displays the **total price**.
- Gracefully handles empty carts with a message.

4. Payment Submission

- Clicking the **“Pay Now”** button triggers processPayment(), which currently shows a confirmation alert.
- Serves as a placeholder for integrating a real payment gateway in the future.

Technologies Used:

- **HTML5**: For structured forms and semantic sections
- **Bootstrap 5**: Used for layout, card components, and responsiveness
- **JavaScript**: DOM manipulation, cart rendering, form handling
- **Local Storage**: Retrieves cart items to display in the summary

```
<!DOCTYPE html>

<html lang="en">



<head>
```

```
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Payment - E-Commerce App</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />
    <style>
        .cart-item {
            border-bottom: 1px solid #ddd;
            padding: 10px 0;
        }

        .cart-item:last-child {
            border-bottom: none;
        }
    </style>
</head>

<body class="bg-light">
    <div id="navbar"></div>
    <div class="container py-5">
        <h2 class="mb-4">Payment Information</h2>

        <!-- Billing Address Form -->
        <div class="card mb-4">
```

```
            <div class="card-header">Billing Address</div>
            <div class="card-body">
                <form id="billingForm">
                    <div class="mb-3">
                        <label for="fullName" class="form-label">Full Name</label>
                        <input type="text" id="fullName" class="form-control" required />
                    </div>
                    <div class="mb-3">
                        <label for="address" class="form-label">Address</label>
                        <input type="text" id="address" class="form-control" required />
                    </div>
                    <div class="mb-3">
                        <label for="city" class="form-label">City</label>
                        <input type="text" id="city" class="form-control" required />
                    </div>
                    <div class="mb-3">
                        <label for="zipCode" class="form-label">Zip Code</label>
                        <input type="text" id="zipCode" class="form-control" required />
                    </div>
                </form>
            </div>
        </div>

        <!-- Payment Method Selection -->
```

```
        <div class="card mb-4">
            <div class="card-header">Payment Method</div>
            <div class="card-body">
                <div class="form-check">
                    <input class="form-check-input" type="radio" name="paymentMethod"
id="creditCard" value="creditCard"
                        checked>
                    <label class="form-check-label" for="creditCard">
                        Credit Card
                    </label>
                </div>
                <div class="form-check">
                    <input class="form-check-input" type="radio" name="paymentMethod" id="paypal"
value="paypal">
                    <label class="form-check-label" for="paypal">
                        PayPal
                    </label>
                </div>
            </div>
        </div>

        <!-- Cart Summary -->
        <div class="card mb-4">
            <div class="card-header">Order Summary</div>
```

```
            <div class="card-body">
                <div id="cartItemsList"></div>
                <div class="d-flex justify-content-between mt-4">
                    <strong>Total:</strong>
                    <span id="totalAmount">$0.00</span>
                </div>
            </div>
        </div>

        <!-- Submit Payment -->
        <button class="btn btn-success w-100" onclick="processPayment()">Pay Now</button>
    </div>

    <script src="./js/api.js"></script>
    <script>
        document.addEventListener('DOMContentLoaded', function () {
            const cart = JSON.parse(localStorage.getItem('cart')) || [];
            const cartItemsList = document.getElementById('cartItemsList');
            const totalAmountElement = document.getElementById('totalAmount');

            if (cart.length === 0) {
                cartItemsList.innerHTML = '<p class="text-muted">Your cart is empty.</p>';
                totalAmountElement.textContent = '$0.00';
                return;
```

```
            }

            let totalAmount = 0;
            cart.forEach(item => {
                const cartItem = document.createElement('div');
                cartItem.className = 'cart-item';
                cartItem.innerHTML = `
              <div class="d-flex justify-content-between">
                <span>Product ${item.id}</span>
                <strong>$${(item.price * item.quantity).toFixed(2)}</strong>
              </div>
              <div class="text-muted">Quantity: ${item.quantity}</div>
            `;
                cartItemsList.appendChild(cartItem);
                totalAmount += item.price * item.quantity;
            });

            totalAmountElement.textContent = `$${totalAmount.toFixed(2)}`;
        });

        function processPayment() {
            alert("Payment Done!");
        }
    </script>
```

```
    </body>

    </html>
```

Page – User Profile:

Purpose:
This page allows both regular users and administrators to view and update user profile information. Regular users can edit their own details, while administrators can manage any user's profile, including assigning roles.

Key Features:

1. Role-Based UI

- **Customer View:**

    - Sees only their own profile fields.
    - Can update name, email, and password.
- **Admin View:**
    - Dropdown to select and edit any user.
    - Can also change the user's role (customer or admin).

2. Editable Form Fields

- Full Name, Email, and Password (all required)
- Role (visible only to admins)
- User ID (stored as a hidden field)

3. Dynamic Behavior

- On page load:
    - Retrieves current user data from localStorage
    - Loads user list if the logged-in user is an admin
- Selecting a user dynamically fills the form with that user's data
- Clicking **"Save Changes"** calls updateUserProfile() to send updated data to the server

Technologies Used:

- **HTML5** – Form structure
- **Bootstrap 5** – Responsive card layout and input styling
- **JavaScript** – Role-based logic, form population, event handling
- **Local Storage** – Tracks current logged-in user
- **URLSearchParams API** – Supports editing a user via ?id=

User Experience Highlights:

- Flexible, clean interface that adapts based on user role
- Pre-filled forms ensure intuitive editing
- Error feedback and success confirmation via alerts

```
<!DOCTYPE html>
```

```
<html lang="en">

<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>User Profile</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />
</head>

<body class="bg-light">
    <div id="navbar"></div>
    <div class="container py-5">
        <h2 class="mb-4" id="pageTitle">Your Profile</h2>

        <div class="card shadow-sm p-4">
            <form id="profileForm">
                <input type="hidden" id="userId" />

                <!-- Admin-only user selector -->
                <div class="mb-3" id="userSelectSection" style="display: none;">
                    <label for="userSelect" class="form-label">Select User</label>
                    <select class="form-select" id="userSelect" onchange="onUserChange()">
                        <option value="">-- Select a User --</option>
```

```
                    </select>
                </div>

                <div class="mb-3">
                    <label for="name" class="form-label">Full Name</label>
                    <input type="text" class="form-control" id="name" required />
                </div>

                <div class="mb-3">
                    <label for="email" class="form-label">Email Address</label>
                    <input type="email" class="form-control" id="email" required />
                </div>

                <div class="mb-3">
                    <label for="password" class="form-label">Password</label>
                    <input type="password" class="form-control" id="password" required />
                </div>

                <div class="mb-3" id="roleSection" style="display: none;">
                    <label for="role" class="form-label">User Role</label>
                    <select class="form-select" id="role">
                        <option value="customer">Customer</option>
                        <option value="admin">Admin</option>
                    </select>
```

```
                </div>

                <button type="button" class="btn btn-primary w-100" onclick="saveProfile()">Save Changes</button>
            </form>
        </div>
    </div>

    <script src="./js/api.js"></script>
    <script>
        const currentUser = JSON.parse(localStorage.getItem("data"));
        const urlParams = new URLSearchParams(window.location.search);
        const editingUserId = urlParams.get("id");
        let usersRes = [];

        async function loadProfile() {
            if (currentUser.role === "admin") {
                document.getElementById("userSelectSection").style.display = "block";
                document.getElementById("roleSection").style.display = "block";

                usersRes = await fetchAllUsers();
                if (usersRes.success) {
                    const userSelect = document.getElementById("userSelect");
                    usersRes.data.forEach(user => {
```

```
                const option = document.createElement("option");
                option.value = user.id;
                option.textContent = `${user.name} (${user.email})`;
                userSelect.appendChild(option);
            });
        }
    }

    if (!editingUserId && currentUser.role !== "admin") {
        fillForm(currentUser);
    } else if (editingUserId && currentUser.role === "admin") {
        const res = await fetchUserById(editingUserId);
        if (res.success) {
            fillForm(res.data);
        } else {
            alert("User not found");
        }
    }
}

function fillForm(user) {
    document.getElementById("userId").value = user.id;
    document.getElementById("name").value = user.name || "";
    document.getElementById("email").value = user.email || "";
```

```
        document.getElementById("password").value = user.password || "";
        if (user.role) {
            document.getElementById("role").value = user.role;
        }
    }

    async function onUserChange() {
        const userId = document.getElementById("userSelect").value;
        if (!userId) return;


        const res = usersRes.data.find(item => item.id == userId)
        if (res) {
            fillForm(res);
        } else {
            alert("User not found");
        }
    }

    async function saveProfile() {
        const userId = document.getElementById("userId").value || currentUser.id;
        const updatedUser = {
            id: userId,
            name: document.getElementById("name").value,
```

```
            email: document.getElementById("email").value,
            password: document.getElementById("password").value,
        };

        if (currentUser.role === "admin") {
            updatedUser.role = document.getElementById("role").value;
        }

        const res = await updateUserProfile(updatedUser);
        if (res.success) {
            alert("Profile updated!");
            if (currentUser.role === "admin") {
                window.location.href = "./welcome.html";
            } else {
                localStorage.setItem("data", JSON.stringify(res.data));
            }
        } else {
            alert("Update failed.");
        }
    }

    document.addEventListener("DOMContentLoaded", loadProfile);
  </script>
</body>
```

```
</html>
```

Page – Register:

Purpose:

The registration page allows new users to sign up for an account on the e-commerce platform by providing their name, email address, and password. This is a critical entry point into the system and connects directly to the back-end user database.

Key Features:

1. User Registration Form

- Inputs:
    - Full Name (required)
    - Email Address (with HTML5 validation)
    - Password (required)
- Built-in browser validation using the required attribute
- Bootstrap's .invalid-feedback used for real-time form validation messages

2. Dynamic Interaction

- Form submission is handled by JavaScript
- Calls registerUser() function from api.js to send data to the server
- Error messages displayed via #registerError for better feedback

3. Navigation

- Redirect link for returning users: "Already have an account? Login here"

Technologies Used:

- **HTML5:** Semantic structure of form inputs
- **Bootstrap 5:** Responsive card-based layout, form styling, and buttons
- **JavaScript:** Form handling, client-side validation logic
- **jQuery (via CDN):** Available for compatibility and future interactivity
- **Custom API File (api.js)**: Handles the actual registration request to the server

User Experience Highlights:

- Minimalist and mobile-friendly design
- Full-screen vertical centering using vh-100
- Clear, easy-to-use interface for new users

```
<!DOCTYPE html>
```

```
<html lang="en">

<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Register - E-Commerce App</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />
    <link rel="stylesheet" href="css/styles.css" />
</head>

<body class="bg-light">
    <div class="container d-flex justify-content-center align-items-center vh-100">
        <div class="card p-4 shadow rounded-4 w-100" style="max-width: 400px;">
            <h2 class="text-center mb-4">Register</h2>
            <form id="registerForm" novalidate>
                <div class="mb-3">
                    <label for="name" class="form-label">Full Name</label>
                    <input type="text" class="form-control" id="name" required />
                    <div class="invalid-feedback">Name is required.</div>
                </div>
                <div class="mb-3">
                    <label for="email" class="form-label">Email address</label>
                    <input type="email" class="form-control" id="email" required />
```

```
                    <div class="invalid-feedback">Please enter a valid email.</div>
                </div>
                <div class="mb-3">
                    <label for="password" class="form-label">Password</label>
                    <input type="password" class="form-control" id="password" required />
                    <div class="invalid-feedback">Password is required.</div>
                </div>
                <button type="submit" class="btn btn-success w-100">Register</button>
                <div id="registerError" class="text-danger mt-2 text-center" style="display:
none;"></div>
            </form>
            <p class="mt-3 text-center">
                Already have an account? <a href="index.html">Login here</a>
            </p>
        </div>
    </div>

    <script src="https://code.jquery.com/jquery-3.7.0.min.js"></script>
    <script src="./js/api.js"></script>

    <script>
        document.addEventListener("DOMContentLoaded", function () {
            const registerForm = document.getElementById('registerForm');
            registerForm.addEventListener('submit', function (e) {
```

```
            e.preventDefault();
            const name = document.getElementById('name').value;
            const email = document.getElementById('email').value;
            const password = document.getElementById('password').value;
            registerUser(name, email, password);
        });
    });
    </script>
</body>

</html>
```

Page – Home (All Products/Catalog):

Purpose:
This page serves as the landing page for logged-in users, displaying all available products in a searchable grid layout. It acts as the main catalog from which customers can browse, search, view details, or add items to their cart.

Key Features:

1. Product Catalog Grid

- Dynamically renders all products using the fetchProducts() function.

- Displays image, name, description, and price.
- Includes conditional buttons depending on user role.

2. Search Functionality

- Real-time filtering of products by name as users type into the search bar.
- Uses a case-insensitive match against product names.

3. Role-Based Controls

- **Admin View:**
    - Can see “Add Product” button
    - Each product card includes **Edit**, **Delete**, **Add to Cart**, and **Product Details** buttons
- **Customer View:**
    - Sees only **Add to Cart** and **Product Details** buttons

4. Graceful Fallbacks

- If no products are available or found, the UI shows user-friendly messages.
- Placeholder image used if a product lacks an image URL.

Technologies Used:

- **HTML5**: Semantic structure
- **Bootstrap 5**: Responsive grid layout, cards, and buttons
- **JavaScript**: DOM manipulation, event handling, data filtering, and routing
- **Local Storage**: Determines user role and manages access

User Experience Highlights:

- Fully responsive, visually clean layout
- Dynamic search provides instant feedback
- Tailored experience for admins vs. customers

```
<!DOCTYPE html>

<html lang="en">


<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>Home - E-Commerce App</title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />

  <link rel="stylesheet" href="./css/styles.css" />

</head>


<body class="bg-light">

  <div id="navbar"></div>


  <div class="container py-5">


    <div class="d-flex justify-content-between align-items-center mb-4">

      <div id="statusMsg" class="text-success mt-2 text-center" style="display: none;"></div>

      <h2>All Products</h2>

      <button id="addProductBtn" class="btn btn-success d-none"
onclick="window.location.href='add-product.html'">Add

        Product</button>

    </div>
```

```
    <div class="mb-4">
      <input type="text" id="searchInput" class="form-control" placeholder="Search products by name..." />
    </div>
    <div id="productGrid" class="row g-4">
      <!-- Products will be rendered here -->
    </div>
  </div>

  <script src="./js/api.js"></script>

  <script>
    let allProducts = []; // Will hold fetched products globally

    document.addEventListener("DOMContentLoaded", async function () {
      const role = JSON.parse(localStorage.getItem('data'));
      const isAdmin = role?.role === 'admin';
      const addBtn = document.getElementById('addProductBtn');
      if (isAdmin && addBtn) {
        addBtn.classList.remove('d-none');
      }

      const res = await fetchProducts();
```

```
        if (res.success && Array.isArray(res.data)) {
          allProducts = res.data;
          renderProducts(allProducts);
        } else {
          document.getElementById('productGrid').innerHTML = '<p class="text-danger">No products available.</p>';
        }

        document.getElementById('searchInput').addEventListener('input', function (e) {
          const searchTerm = e.target.value.toLowerCase();
          const filtered = allProducts.filter(product => product.name.toLowerCase().includes(searchTerm));
          renderProducts(filtered);
        });
      });

      function renderProducts(products) {
        const productGrid = document.getElementById('productGrid');
        productGrid.innerHTML = '';

        if (products.length === 0) {
          productGrid.innerHTML = '<p class="text-danger">No matching products found.</p>';
          return;
        }
```

```
        const isAdmin = JSON.parse(localStorage.getItem('data'))?.role === 'admin';

        products.forEach(product => {
          const col = document.createElement('div');
          col.className = 'col-md-4';
          col.innerHTML = `
                <div class="card h-100 shadow-sm rounded-4">
                    <img src="${product.image_url || 'https://via.placeholder.com/300x200'}"
class="card-img-top" alt="${product.name}">
                    <div class="card-body d-flex flex-column">
                        <h5 class="card-title">${product.name}</h5>
                        <p class="card-text text-muted">${product.description || 'No description'}</p>
                        <div class="mt-auto d-flex justify-content-between align-items-center">
                            <strong>$${product.price}</strong>
                            ${isAdmin ?
              `<div class="btn-group">
                                    <button class="btn btn-sm btn-warning"
onclick="editProduct('${product.id}')">Edit</button>  
                                    <button class="btn btn-sm btn-danger"
onclick="deleteProduct('${product.id}')">Delete</button>   
                                    <button class="btn btn-sm btn-primary"
onclick='addToCart(${JSON.stringify(product)})'>Add to Cart</button>  
                                    <button class="btn btn-sm btn-primary"
onclick='productDetail(${product.id})'>Product Details</button>
                                </div>`
```

```
            :

            `<div class="btn-group">

                        <button class="btn btn-sm btn-primary"
onclick='addToCart(${JSON.stringify(product)})'>Add to Cart</button>  

                        <button class="btn btn-sm btn-primary"
onclick='productDetail(${product.id})'>Product Details</button>

                        </div>`

          }

                    </div>

                  </div>

                </div>`;

        productGrid.appendChild(col);

      });

    }

    function viewCart() {

      window.location.href = './cart.html';

    }

    function productDetail(id) {

      window.location.href = `./item-description.html?id=${id}`;

    }

    function editProduct(id) {
```

```
        window.location.href = `edit-product.html?id=${id}`;
    }
  </script>

</body>

</html>
```

Challenges & Resolutions:

Challenge	Resolution
1. Responsive Layout Across Devices Maintaining consistent layout and spacing across multiple screen sizes.	Used **Bootstrap 5's grid system** (`container`, `row`, `col-md-*`, `vh-100`, etc.) and **responsive utility classes** to ensure mobile-first design.

2. Handling Role-Based Access (Admin vs Customer) Showing/hiding elements like Add/Edit/Delete buttons based on user roles.	Retrieved user role from `localStorage` and used JavaScript conditions to dynamically show/hide elements (e.g., `addProductBtn`, admin-specific buttons).
3. Dynamic Content Rendering Rendering lists of products, cart items, or user data from the backend or local storage.	Used `fetch*()` functions and `forEach()` loops to inject dynamic HTML into target containers (e.g., `productGrid`, `cartItems`, `userSelect`).
4. Form Validation (Client-Side) Preventing form submission without valid inputs.	Utilized HTML5 attributes (`required`, `type="email"`, etc.) along with Bootstrap's `invalid-feedback` elements and custom JS for additional checks.
5. Managing State with `localStorage` Ensuring user and cart data persisted between page reloads.	Stored and retrieved `user` and `cart` data using `localStorage.getItem()` / `setItem()` for smoother session handling without login repetition.
6. Updating Existing Data (Edit Product/Profile) Pre-filling forms with existing data for update operations.	Parsed URL parameters using `URLSearchParams`, fetched data with `fetchById()` functions, and auto-filled forms using JavaScript.
7. Handling Empty States Gracefully Cart or product list being empty caused poor UI or errors.	Added conditional checks and fallback messages (e.g., "Your cart is empty", "No matching products found") to improve UX.

8. Ensuring Code Reusability Across Pages Reducing repetition of logic like rendering product cards or validating forms.	Modularized functionality into reusable functions (e.g., `renderProducts()`, `addToCart()`, `fillForm()`) stored in `api.js`.
9. Navigating Between Pages With Data Redirecting with proper context (e.g., product ID or user ID).	Used query parameters in URLs (`productDetail(id)`, `edit-product.html?id=...`) and read them using `URLSearchParams`.
10. Providing Feedback After Actions Form submissions or deletions lacked confirmation initially.	Added simple feedback messages using `alert()`, `#statusMsg`, or redirect logic (e.g., after saving a profile or completing payment).

Back-End Development:

Objective:

The goal of Task 2 was to implement the back-end functionality of the e-commerce web application using Node.js and Express. The server was designed to manage data operations, handle user authentication, and support interaction between the front-end and the MySQL database.

Main Components:

1. app.js

- Entry point of the back-end application.

- Initializes Express, middleware, database connection, and routes.

2. Database Configuration (db/config.js)

- Uses mysql2 to connect to a MySQL database.
- Connection values are loaded from a .env file.

3. Environment Variables (.env)

- Stores database credentials, JWT secret, and port number.
- Example:

DB_HOST=localhost

DB_USER=root

DB_PASS=password

DB_NAME=electronics_store

JWT_SECRET=supersecretkey

4. Controllers

- Logic for handling routes and database operations.

userController.js

- Handles user registration, login, and profile updates.
- Implements password hashing (bcryptjs) and JWT token generation.

productController.js

- CRUD operations for products.
- Admin-only access to create, update, and delete products.

cartController.js

- Manages user's shopping cart: add, remove, update cart items.

orderControllers.js

- Handles order placement and order history storage.

5. Middleware (authMiddleware.js)

- Validates JWT tokens.
- Protects routes requiring authentication.

API Routes Overview:

Examples of routes implemented:

- POST /api/register – Register a new user.
- POST /api/login – Authenticate and return JWT.
- GET /api/products – Retrieve all products.
- POST /api/products – Add product (admin only).
- PUT /api/products/:id – Update product.
- DELETE /api/products/:id – Delete product.
- POST /api/cart – Add to cart.
- POST /api/orders – Place an order.

Form Validation & Error Handling:

- Server-side validation using custom logic and the validator library.
- Sends appropriate HTTP status codes and error messages.
- Protects database from malformed or malicious input.

Session and Authentication:

- JWT tokens used for login sessions.
- Tokens stored on the client side and verified on protected routes.
- Middleware ensures only authenticated users can access restricted endpoints.

Conclusion:

The back-end implementation of the Electronics Store successfully connects the front-end interface to a secure, data-driven server. It uses Express routing, middleware, and database interaction patterns that follow modern web development practices. This layer is critical for handling dynamic interactions, secure logins, and data persistence across the application.

Database Design and Implementation:

Objective:

The objective of Task 3 was to design and implement a well-structured relational database using MySQL to support the functionality of the Electronics Store web application. The database enables features such as user authentication, product listings, shopping cart operations, and order processing.

Entity-Relationship Design (ERD):

The ERD includes four core entities:

- **Users**

- **Products**
- **Cart**
- **Orders**

Each table is connected through primary and foreign key relationships, ensuring data integrity and referential linkage.

Entity Relationships:

- A user can place many orders → (1:M)
- A user can have many cart items → (1:M)
- Each order is linked to one product and one user
- The cart table serves as a temporary junction between users and products

Table Breakdown:

1. Users Table

- Stores user information including role (admin/customer)
- Has a unique email constraint
- Primary key: id

sql

CopyEdit

```
CREATE TABLE users (
  id INT(11) NOT NULL AUTO_INCREMENT,
  name VARCHAR(100),
  email VARCHAR(100) UNIQUE,
  password VARCHAR(255),
  created_at TIMESTAMP DEFAULT current_timestamp(),
  role VARCHAR(20) DEFAULT 'customer',
  PRIMARY KEY (id)
```

);

2. Products Table

- Stores product details such as name, price, category, and image
- Primary key: id

sql

CopyEdit

```
CREATE TABLE products (
  id INT(11) NOT NULL AUTO_INCREMENT,
  name VARCHAR(150),
  description TEXT,
  price DECIMAL(10,2),
  image_url VARCHAR(255),
  stock INT(11),
  category VARCHAR(100),
  PRIMARY KEY (id)
);
```

3. Cart Table

- Stores temporary user selections before purchase
- Foreign keys: user_id, product_id
- Enforced with ON DELETE CASCADE for cleanup

sql

CopyEdit

```
CREATE TABLE cart (
  id INT(11) NOT NULL AUTO_INCREMENT,
  user_id INT(11),
```

```
  product_id INT(11),
  quantity INT(11),
  PRIMARY KEY (id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
);
```

4. Orders Table

- Records completed purchases
- Includes quantity and total price
- Linked to a user and product

```
sql
CopyEdit
CREATE TABLE orders (
  id INT(11) NOT NULL AUTO_INCREMENT,
  user_id INT(11),
  product_id INT(11),
  quantities INT(11),
  total_price DECIMAL(10,2),
  order_date TIMESTAMP DEFAULT current_timestamp(),
  PRIMARY KEY (id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
```

);

Conclusion:

Overall Learning Experience:

This project offered a comprehensive, hands-on experience in full-stack web development. By designing and implementing both the front-end and back-end of an e-commerce application, I deepened my understanding of how modern web systems function in practice. The integration of client-side interfaces with a server-side API and a relational database mirrored the architecture used in real-world applications.

Technologies I Became Confident With:

- **MySQL**: I became more confident in writing SQL queries, designing normalized tables, and using foreign key constraints for relational integrity.
- **Node.js & Express.js**: I gained practical experience building RESTful APIs and handling middleware, authentication, and route structuring.
- **JavaScript (ES6+)**: Improved my skills in manipulating the DOM, handling events, and working with browser storage (e.g., localStorage).
- **Bootstrap 5**: Helped me build responsive, clean, and mobile-friendly layouts without needing custom CSS from scratch.
- **JWT & bcrypt**: Implemented secure login systems using token-based authentication and password hashing.

Real-World Application Potential:

The Electronics Store application could easily be extended into a real-world product by:

- Connecting to a payment gateway like Stripe or PayPal
- Adding email confirmation and password reset
- Introducing inventory tracking, review systems, and admin dashboards
  It lays the groundwork for scalable e-commerce or inventory-based platforms.

Summary of the Whole Process:

1. **Task 1** focused on creating the front-end using HTML, Bootstrap, and JS.
2. **Task 2** handled back-end development with Node.js, Express, and REST APIs.
3. **Task 3** involved designing and implementing a normalized MySQL database with clear relationships and constraints.
   Throughout the process, I implemented role-based access, dynamic rendering, and secure user authentication.

Reflection: What Worked Well vs What Could Be Improved:

What Worked Well:

- Front-end and back-end integration was seamless using RESTful APIs.
- Role-based interface adaptation (admin vs customer) worked reliably.
- Database constraints like ON DELETE CASCADE helped maintain referential integrity.

What Could Be Improved:

- Error handling could be more user-friendly (e.g., using toasts instead of alert()).
- Back-end validation can be extended with libraries like Joi or express-validator.
- Form UX could be improved with better inline validation and auto-filling on edit screens.
- A more modular file structure and route separation would help scale the codebase.